НЕ ФИНАЛЬНАЯ ВЕРСИЯ — WORK IN PROGRESS

Лекция 03.09.2026

ISO/ IEC 2382:2015 Данные - поддающиеся многократной интерпретации представление информации в формализованном виде пригодном для передачи, интерпретации или обработки Quotation: Data - reinterpretable representation of information in a formalized manner suitable for communication, interpretation, or processing

Лекция 10.09.2026

Фундаментальные противоречия: CAP теорема, R&W (Чтение и запись).

ANSI / SPARC выделил три уровня(Point of View) архитектуры:
- внешний
- концептуальный
- внутренний (физический уровень)

Типы моделей данных:
- модель Сущность-связь (Enity-Relation) | внешний + концептуальный уровни
- логическая модель (logical schema) | чистый концептуальный уровень
- физическая модель (physical schema) | внутренний + концептуальный уровни

Модель сущность-связь (ER-diagram)
Для построения модели необходимо выделить:
1) сущность
   a) Слабые и сильные.
2) свойство (атрибут)
   a) Простые и составные.
   b) Обязательные и необязательные.
   c) Однозначные и многозначные.
   d) Домен (область допустимых значений).
3) связь
   a) 1 к 1
   b) 1 ко многим
   c) многие к многим

Принципиальная разница между Реляционной БД и Постреляционной БД - в реляционной все атрибуты неделимы, в постреляционной — все многозначные.

Нотация Чена - наиболее распространенный вид записи. Другие подходы: Воронья лапка, IDF1X...

Модель сущность-связь не привязывается к логической модели.

Лекция 17.09.2026

Логическая модель

1) Иерархическая модель данных. (конец 50-х)
   - Сегменты данных.
2) Сетевая модель данных (графовая).
3) Реляционная модель данных.

   Важная разница: в иерархической модели все связи - в памяти, при помощи указателей, в то время как реляционная состоит на парах идентификаторов. За это платим потерей целостности (ответственность перекладывается на пользователя).
4) Постреляционная модель.
5) Многомерная модель
   - OLAP-куб (к примеру построение по товар/регион/продавец и в клетках количество товаров). Алгебра топологий...
   - По целостности данная модель данных «разорвана». Удобна для аналитиков.
6) Объектная модель.
   - ORM-фреймворки (Object–relational mapping).
   - Проблема консистентности класса.
   - Идея сериализации объекта (возможно с подписью), атрибуты в поля, но они read only. Для write нужна десериализация.

Лекция 24.09.2026

Глубокое погружение в Реляционную модель.

Автор математической модели — Эдгар Франк Кодд. Терминология:

- Relation — таблица (массив в терминологии 80-х)
- Строка таблица — кортеж (но кортеж это не упорядоченный)
- Столбец — атрибут
- Строка заголовков — схема отношения
- Поле (ячейка) — значение конкретного атрибута для конкретного кортежа
- Домен — множество допустимых значений атрибута.
- Степень отношения — количество атрибутов (столбцов).
- Кардинальность отношения — количество кортежей (строк).

Реляционная алгебра — раздел математики, не ограничивающийся возможностью реализации. Поэтому в реляционной алгебре допустимы к примеру домены из множества всех натуральных чисел, в то время как в реализации такое невозможно - нельзя допустить бесконечное количество значений.

Пусть $D_1, D_2, D_3 ... D_n$. Тогда $R =< d_1, d_2, d_3, ..., d_n >,\ d_i \in D_i$

Однако Кодд определяет отношение, как обладающее еще 6 свойствами:

1) Каждое поле содержит ровно одно неделимое значение (атомарность поля)
   - Конфликт: хранимое значение будет делимо (например ФИО — это строка символов) из-за деталей реализации, более того: LIKE — полное нарушение этого свойства.
2) В отношении каждый кортеж должен быть уникальным (все строки различны).
   - Конфликт: проекция. Если в таблице ИСУ-ФИО-НОМЕРГР мы возьмем ФИО-НОМЕРГР, у нас могут быть однофамильцы, различить которых можно только по ИСУ. То есть уникальность отношения нарушится. По Кодду - SELECT может быть только SELECT DISTINCT.
3) Уникальность имени отношения в реляционной схеме (совокупность всех таблиц)
   - Конфликт: Слияние двух реляционных схем. Например при объединении организаций, которым нужно интегрировать БД. Если взять имя как полное имя (через точку) — свойство соблюдается.
4) Уникальность имени атрибута в пределах отношения.
   - Конфликт: JOIN. Таблица vendor имеет name, и таблица product имеет name. С точки зрения Реляционной алгебры — это больше не отношение.
5) Значения атрибута берутся из одного и того же домена.
   - Конфликт: допустим доменом для группы у студента будет номера групп в таблице группы. Если мы удалим одну из групп в таблице групп — мы нарушим свойство для таблицы студентов. Варианты разрешения по Кодду:
     - Жертвы студентов богу Машине Базы Данных. **Удалить** всех студентов группы.
     - Удаление атрибута у студентов группы. но нельзя создать пустую память — костыль null. NULL ломает половину алгебры, потому что требуется делать оговорки почти в каждом определении.
6) Порядок следования атрибутов и порядок следования кортежей не имеют значения.
   - Конфликт: ORDER BY. Более того — ORDER BY 2 также работает, то есть еще и атрибуты упорядочены. pressF.

Тем не менее, реализацию (SQL) Кодд ненавидел — инженеры упростили его модель, нарушив многие свойства, и фактически сломав мат выкладки по оценке асимптотик.

Более того, нельзя однозначно построить соответствие между уровнями моделей - логической Рел. алгеброй и моделью Сущность-Связь.Не любое отношение является сущностью: как пример студент принадлежащий двум группам. Это связь, не сущность. Или сущность как композиция отношений — например если все атрибуты разделены по разным отношениям.

Виды ключей:
- Суперключ. Атрибут или множество атрибутов единственным образом идентифицирующий кортеж. Из свойства отношений — любое отношение имеет хотя бы один суперключ.
- Потенциальный ключ. Суперключ, который не содержит подмножество также являющегося супреключем данного отношения.

Иногда говорят что потенциальный ключ это суперключ с мин количеством атрибутов. Это не правда. Контрпример: Два потенциальных ключа. Отношение из ису-номера и серии+номера паспорта (разные столбцы). Серия+номер являются потенциальным ключем,

также как и ису, они оба супреключи, но разной длинны. По определению через минимальный набор атрибутов, потенциальный ключ только ису. Однако серия+номер паспорта тоже потенциальный ключ как мы выяснили. Противоречие.

- Первичный ключ. Потенциальный ключ который выбран для уникальной идентификации кортежа внутри отношения.
- Внешний ключ. Атрибут или множество атрибутов внутри отношения, которые соответствуют потенциальному ключу некоторого, может быть того же самого, отношения.

У отношения есть целостность, которые бывают:

- Сущностной. В базовом отношении ни один элемент первичного ключа не может содержать NULL-значения.
- Ссылочной. Ссылочная целостность - если в отношении существует внешний ключ, то значение внешнего ключа должно либо соответствовать полностью значению потенциального ключа, либо внешний ключ должен полностью состоять из NULL-значений.

Лекция 01.10.2026

Операции рел. алгебры.

По Кодду: Проверка принадлежности через соответствие доменов атрибутам. Главные задачи создания именно алгебры:
1) Универсальный формализм без иных трактовок, чтобы реализации СУБД были совместимы, т.е универсализация.
2) Предсказуемая асимптотика

Мир если бы сортировки были за O(n) (meme)
3) Вопрос — нельзя ли бинарно (вероятностно по Баесу) определить нарушится ли целостность данных. Оценка целостности данных после операции.

Реляционная алгебра замкнута. Операции делятся на унарные и бинарные. Рассмотрим унарные:
1) Проекция $\prod_{a_1,a_2,\ldots,a_n}(R)$. Проекция создает вертикальное подмножество нового отношения, создаваемая посредством извлечения значения указанных атрибутов и исключения из результата строк дубликатов. = SELECT. НО! уникальность означает что по определению проекции, может существовать только SELECT DISTINCT.
2) Выборка $\sigma_{\text{предикат}}(R)$ Определяет результирующее отношение, которое содержит только те кортежи исходного отношения, которые удовлетворяют определенному предикату. = WHERE. Формально LIKE нарушает отношение.

Бинарные
1) Объединение $R \cup S$. .Объединение двух отношений R и S определяет новое отношение, которое включает все кортежи содержащиеся только в R, все кортежи содержащиеся только в S, и кортежи содержащиеся в обоих отношениях сразу, с исключением дубликатов.Здесь возникает понятие — совместимость по объединению. R и S совместимы по объединению если они состоят из одинакового количества атрибутов, и эти атрибуты попарно имеют общие домены.

2) Разность $R - S$. Разность двух отношений состоит из кортежей которые имеются в отношении R но отсутствуют в отношении S.
3) Пересечение $R \cap S$. Определяет отношение, которое содержит только кортежи, принадлежащие и R, и S. Обычно стараются избегать операций $\cup, \cap, -$, и дубликации кортежей в отношениях.
4) Декартово произведение $R \times S$. Операция определяет новое отношение, которое является результатом конкатенации каждого кортежа из отношения R с каждым кортежем отношения S.
5) Тета-соединение $R \underset{F}{\bowtie} S$. $F = Ra_i$. Тета-соединение содержит кортежи из декартового произведения, удовлетворяющие предикату F, где F имеет вид:

   $$F = R.a_i \Theta S.b_i$$

   причем в этом случае Θ — операция попарного сравнения, то есть операция возвращая True/False для каждого значения атрибутов. $\Theta \in \{=, \neq, <, >, ...\}$

   Идейно это JOIN в SQL, причем JOIN строго соответствует Тета-соединению.
6) Экви-соединение. Обозначается также как Тета-соединение, но мы четко определяем что $\Theta =$ "=", то есть это строго отношение по равенству.
7) Естественное соединение $R \bowtie S$. Это Экви-соединение двух отношений, выполненное по всем общим атрибутам, из результатов которого исключается по одному экземпляру каждого общего атрибута.

   По Кодду невозможно взять несуществующего студента, потому что мы можем взять только значение из домена. Контрпример: выдуманный студент, у которого все атрибуты удовлетворяют доменам. Но семантически же он не существует. Операция на которой разругались Кодд и IBM

   Еще пример: Две таблицы, соедини по всем общим атрибутам (например по номеру ису, таблица студентов и таблица менторов). Но в этой ситуации мы должны каким то образом провалидировать что isu номер там и там есть одинаковые вещи, а не случайный набор чисел / не разные по семантике домены. (Например id и id в двух таблицах)
8) Левое внешнее соединение $R \bowtie S$. Естественное соединение при котором в результирующее отношение включаются также кортежи отношения R, не имеющие совпадающих значений в общих столбцах отношения S. В реальности LEFT JOIN ближе к тета-соединению. В реализации есть NULL значения, которые катастрофа по Кодду.
9) Правое внешнее соединение $R \bowtie S$
10) Полное внешнее соединение $R \bowtie S$

Следует разделять $\cup$ объединение и $\bowtie$ соединение

Лекция 08.10.2026

Structured Query Language (SQL) — *декларативный язык* программирования, применяемый для создания, модификации и управления данными в реляционной базе данных, управляемой соответствующей системой управления базами данных.

В стандартном SQL 4 базовых оператора:

1) SELECT (опеределение из доки Postgre)
2) INSERT
3) UPDATE
4) DELETE

При переходе к SQL корректнее называть отношения таблицами, потому что строго говоря таблица в SQL не является отношением по Кодду.

> *“SQL — страшный язык”*
>
> ***— Маятин А.В.***

Порядок выполнения SELECT:

1) FROM
2) WHERE
3) GROUP BY HAVING
4) SELECT
5) DISTINCT
6) ORDER BY

Эффективное использование агрегатов – отсечение при помощи WHERE большей части ненужных строк, и лишь потом GROUP BY.

> *“А нехрен было в академ уходить”*
>
> ***— Маятин А.В.***

Наивный JOIN реализуется тривиально, Nested loop:

```
for r in R:
  for s in S:
    if P(R.r, S.s):
      print(r+s)
```

Второй способ, важное замечание: R.a — PrimaryKey, S.b — ForeignKey.Второй способ почти всегда быстрее первого.

```
R.sort(a)
R.sort(b)
while not endof(R) and not endof(S)
  if R.a < S.b:
    next(R)
  if R.a > S.b:
    next(S)
  if R.a == S.b
    print(r+s)
    next(S)
```

Третий способ — Вложенные подзапросы. По времени выигрыша нет, но вот по памяти это эффективнее чем второй подход.

Важно понимать, что много вещей в SQL полагаются на адекватность пользователя.

Лекция 15.10.2026

Лекции не было.

Лекция 22.10.2026

Избыточность и дублирование

Дублирование нормально, данные нельзя получить из другого источника. Избыточность не нормальна, приводит к *аномалиям*.

Виды аномалий:
1) Аномалия модификации — изменение значения одной записи повлечет за собою просмотр всей таблицы и изменение некоторых других записей
2) Аномалия удаления — при удалении записи может пропасть другая информация.
3) Аномалия добавления (вставки) — информацию в таблицу нельзя поместить пока она не полна или требуется дополнительный просмотр таблицы.

Решением аномалий является декомпозиция отношений.

Функциональная зависимость атрибутов

В отношении R атрибут Y функционально (полно) зависит от атрибута X тогда и только тогда, когда каждому значению X соответствует в точности одно значение Y. X в этой ситуации называется *детерминантом*, а Y — *зависимая часть*.

Частичная функциональная зависимость это зависимость не ключевого атрибута от части составного потенциального ключа.

Транзитивная функциональная зависимость это функциональная зависимость $X \to Z$, такая что $\exists Y : X \to Y \land Y \to Z$.

Нормальные формы

> **1-НФ**
>
> Отношение находится в 1-НФ если все его атрибуты простые, то есть имеют единственное значение

> **2-НФ**
>
> Отношение находится в 2-НФ если оно находится в 1-НФ и каждый не ключевой атрибут функционально полно зависит от первичного ключа.

3-НФ

Отношение находится в 3-НФ если оно находится в 2-НФ и все не ключевые атрибуты взаимно независимы.

или

Отношение находится в 3-НФ если оно находится в 2-НФ и ни один не ключевой атрибут не находится в транзитивной функциональной зависимости от потенциального ключа.

НФБК

Отношение находится в нормальной форме Бойса-Кодда если оно находится в 3-НФ и детерминанты всех зависимостей являются потенциальными ключами.

Фактически, избегать ситуаций с пересечением потенциальных ключей.

4-НФ

Отношение находится в 4-НФ если оно находится в НФБК и не содержит нетривиальных многозначных зависимостей.

Старшие НФ ловят аномалии, которые сложно доказать как аномалию. Фактически, это может быть частью бизнес логики. 4-6НФ не выносятся на экзамен/рубежку.

Лекция 26.10.2026 (вместо Лекции 15.10.2026)

Производительность. Индексы

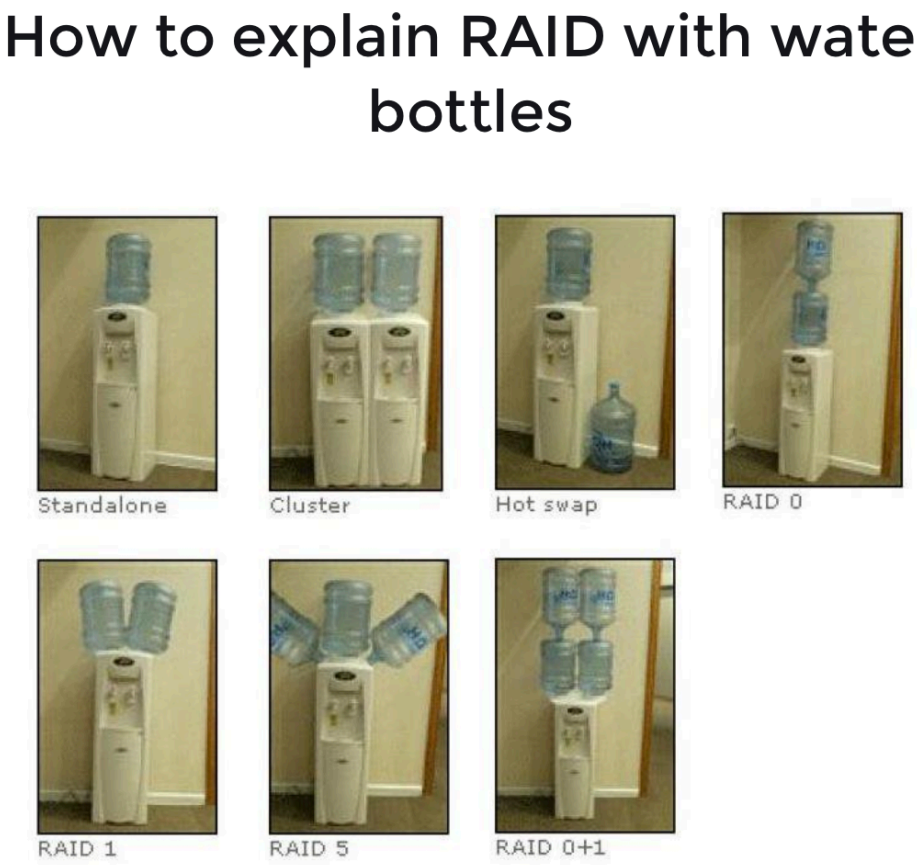
Лекция 29.10.2026

!Рубежка! Проходит в формате развернутых ответов на вопросы на физическом органическом носителе

Лекция 05.11.2026

Надежность. Невозможно создать абсолютно надежную систему.

Физическая надежность.
Объяснение системы RAID:



Следующий уровень — Система Хранения Данных (СХД). Фактически — RAID с более сложной структурой, учитывающей скорость дисков, их количество (десятки), автобалансировкой.

Уровень выше — Дата Центр (Привет ЦОДы). Целые здания, в которых в сотни СХД и дисков обрабатывают и сохраняют огромное количество данных. Дата-центры разделяют на тиры, где TIER4 — самый лучший.

Логическая надежность.

ID	Count
001	100

123	300
345	200

При попытке перевода со счета 001 на счет 123, может произойти сбой электричества, и тогда:

- Если мы сначала сняли 100 со счета 001, то 100 единиц уйдет??? (в пользу банка получается).
- Если мы сначала добавили 100 к счету 123, то 100 единиц уйдет (в минус банку).

Решение: Транзакция — последовательность действий с базой данных,в которой либо все действия выполняются успешно (COMMIT), либо не выполняется ни одно из них (ROLLBACK).

Транзакция должна соответствовать 4м свойствам ACID:

1) Атомарность (Atomicity): Транзакция выполняется либо полностью, либо не выполняется вообще. Промежуточных результатов не бывает.
2) Согласованность (Consistency): База данных всегда переходит из одного корректного состояния в другое, не нарушая правила системы.
3) Изоляция (Isolation): Если запущено несколько конкурирующих транзакций (касаются одного сегмента данных), любое обновление состояния БД выполненное одной из них не может повлиять на результат выполнения другой.
4) Устойчивость (Долговечность | Сохранность) (Durability): Когда транзакция завершена, ее результаты обновления сохранятся даже если в следующий момент произойдет сбой

Isolation

Существует 4 проблемы конкурирующих транзакций, и соответственно, 4 уровня изоляции.

Проблема потерянного обновления

Одна из записей затирает другую. Все проблемы транзакций решаются блокировками. Блокировки бывают монопольные (блокируются все виды доступа) и коллективные | кооперативные (блокируется только изменение).

Isolation level 1 — незавершенное чтение. Фактически требует чтобы изменять данные могла только одна транзакция. Производительность очевидно упала.

Проблема грязного чтения (Dirty reading)

Возникает когда при чтении одной транзакции кортежа оказывается, что он изменен, но еще не сохранен другой не завершившейся транзакцией, которая затем будет отменена.

Пример: транзакции +200 из 001 в 123 и +500 из 123 в 345. Если первая транзакция сначала добавила 200 к 123, и пошла вычитать их из 001 (что приведет к ROLLBACK ведь там нет 200), вторая транзакция в этот момент прочитает 123, увидит 500, и переведет без ROLLBACK.

Isolation level 2 — завершенное чтение. Если в одной из транзакций ставится блокировка на ячейку которую планируется изменять, эту ячейку запрещается и читать, и изменять до завершения транзакции COMMIT/ROLLBACK. Производительность еще раз очевидно упала.

Проблема неповторяемого чтения

При повторном чтении данных, уже считанных ранее, транзакция обнаруживает модификацию, вызванную другой, уже завершенной, транзакцией.

Пример: коррелированный вложенный подзапрос, фактически цикл. Пусть хотим начислить проценты на категорию пользователей. В момент первого прохода мы начислили ячейке один проценты. За время пока мы до нее снова дойдем, может произойти транзакция чтения-изменения этой конкретной ячейки. И тогда при возврате к этой ячейке основным циклом, она попадет и в другую категорию, и ей будет еще раз начислен процент.

Isolation level 3 — воспроизводимое чтение. Если транзакция считывает данные, то никакая другая транзакция не может изменять затронутые данные. Производительность еще больше упала.

Проблема фантномного чтения

При повторном чтении данных, транзакция обнаруживает новые кортежи, добавленные другой транзакцией, завершенной после предыдущего чтения данных.

Пример: первое чтение посчитало сумму заказа, второе должно добавить стоимость заказа, поле которого еще не существует.

Isolation level 4 — сериализуемость. Если транзакция обращается к данным (любое обращение), то никакая другая транзакция не сможет добавить новые или изменить существующие строки которые могут быть считанны при выполнении транзакции. Проблема производительности настолько серьезна, что данный уровень почти не используется.

> *“Мог(н)ут, не мог(н)ут, кто его знает”*
>
> ***— Маятин А.В.***

Durability

С одной стороны мы хотим кешировать данные в RAM чтобы не ходить постоянно в диск. С другой стороны хотим обеспечить устойчивость данных к сбоям.

Журналирование транзакций — сначала пишется план транзакции, причем план пишется в независимую память. Далее выполняем действие транзакции — стираем строчку в журнале. Тогда при сбое можно восстановить и довыполнить транзакцию.

Лекция 12.11.2026

Безопасность